PROJECT PLAN

Smart Air Quality Monitoring System Based on IoT & AI

Smart Air Quality Monitoring System using IoT & AI

COURSE: Advanced IoT Solutions and Systems

Instructor:	Nguyen Chien Trinh
Team Members:	Nguyen Ngoc Duong — B22DCVT112 Phung Dinh Dat — B22DCVT133 Tran Van Huy — B22DCVT246

Hanoi, 2026

Contents

1 Project Overview **2**
1.1 Background & Objectives 2

2 System Architecture & Data Flow **3**
2.1 Overall Architecture Diagram 3
2.2 Data Flow 3

3 MQTT Design & Hardware **4**
3.1 MQTT Topics & Payload 4
3.2 Hardware Diagram 4

4 Software & AI Service **5**
4.1 FreeRTOS Tasks on ESP32 5
4.2 AI Service Pipeline 5
4.3 Old vs New Architecture Comparison 5

5 Implementation Plan **6**
5.1 Task Assignment 6
5.2 8-Week Timeline 6
5.3 Conclusion 6

Chapter 1

Project Overview

1.1 Background & Objectives

Air pollution poses serious threats to public health, yet most existing monitoring systems are expensive and lack intelligent analysis capabilities. This project proposes a **low-cost, real-time, AI-integrated** IoT system suitable for homes, offices, and small industrial environments.

Main Objectives

1. ESP32 node continuously collects: temperature, humidity, CO, VOC, and fine dust PM2.5.
2. Transmit data to the cloud via MQTT, near real-time (every 2–5 seconds).
3. AI Service: anomaly detection, trend forecasting, natural language alerts.
4. Instant alert feedback to ESP32 via MQTT (retain + QoS1).
5. Visual display on onboard TFT screen and web dashboard.

In scope: 1 ESP32 node + 4 sensors; cloud MQTT + AI; TFT display; web dashboard.
Out of scope: Wide-area multi-point deployment, video surveillance, native mobile app.

Chapter 2

System Architecture & Data Flow

2.1 Overall Architecture Diagram

2.2 Data Flow

A. Local	Sensors → ESP32 → TFT + localRules() → Buzzer/LED (no network dependency)
B. Stream	ESP32 $\xrightarrow{\text{MQTT tele}}$ Broker → AI + InfluxDB (every 2–5 seconds)
C. Snapshot	ESP32 $\xrightarrow{\text{snap/10min}}$ Broker → InfluxDB (avg/min/max every 10 minutes)
D. Alert	AI $\xrightarrow{\text{cmd retain QoS1}}$ Broker → ESP32 (red screen, relay ON)

Chapter 3

MQTT Design & Hardware

3.1 MQTT Topics & Payload

Topic	Direction	Purpose	Frequency
`tele/<id>/sensors`	ESP32→Cloud	Continuous telemetry stream	2–5 sec
`snap/<id>/10min`	ESP32→Cloud	Statistical snapshot avg/min/max	10 min
`cmd/<id>/alert`	Cloud→ESP32	AI alert (retain=true, QoS1)	On detection
`cmd/<id>/relay1`	Cloud→ESP32	Relay ON/OFF control	On command

Table 3.1: MQTT Topics Summary

Sample payload (`tele/<id>/sensors`):


```
{"device_id":"esp32-01","ts":1760000000,"temp_c":29.4,"hum_pct":72.1,
"mq135_adc":1840,"mq7_adc":2100,"dust_adc":980,"rssi":-61}
```


3.2 Hardware Diagram

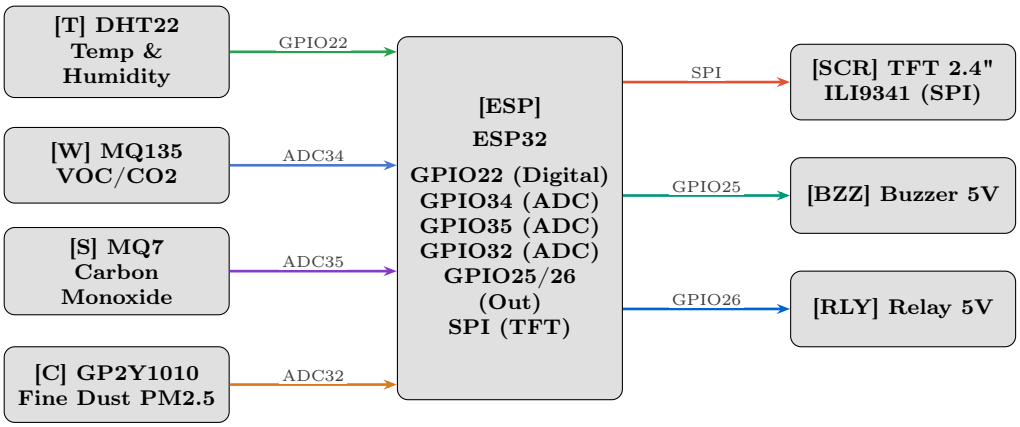

Figure 3.1: ESP32 hardware connection diagram

No.	Component	Function	Qty	Interface
1	ESP32 DevKit v1	Main microcontroller, WiFi	1	—
2	DHT22	Temperature & humidity	1	Digital
3	MQ-135	VOC, CO2 gas	1	ADC
4	MQ-7	Carbon monoxide (CO)	1	ADC
5	GP2Y1010AU0F	Fine dust PM2.5	1	ADC
6	TFT 2.4" ILI9341	Display screen	1	SPI
7	Buzzer + Relay 5V	Audio alert, device control	2	GPIO

Table 3.2: Hardware component list

Chapter 4

Software & AI Service

4.1 FreeRTOS Tasks on ESP32

Task	Priority	Cycle	Function
readSensors	HIGH	50ms	Read ADC, noise filter, attach NTP timestamp
localRules	HIGH	100ms	Buzzer/LED alert without network dependency
subscribeCMD	HIGH	Event	Receive AI alerts, store in AlertBuffer
drawScreen	NORMAL	500ms	Render TFT: values, mini chart, status color
publishTele	NORMAL	2–5s	Send MQTT telemetry stream
publishSnap	LOW	10min	Send avg/min/max snapshot

store&forward: Buffer stores data when offline and retransmits upon reconnection.

4.2 AI Service Pipeline

1. **Subscribe** to topic `tele/<id>/sensors`, parse JSON payload.
2. **Sliding window** of 30 data points (2.5–5 minutes), updated continuously.
3. **Anomaly detection:** Z-score/IQR/LSTM detects sudden spikes.
4. **Forecasting:** ARIMA/moving-average predicts trends for the next 30 minutes.
5. **Reasoning:** LLM summarizes the situation and suggests specific actions.
6. **Publish:** `cmd/<id>/alert` with retain=true, QoS1.

4.3 Old vs New Architecture Comparison

Criterion	Old (Firebase HTTP)	New (MQTT)
Protocol	HTTP REST	MQTT lightweight
Send frequency	Every 10 minutes	Every 2–5 seconds (stream)
Alert latency	$\leq$ 10 minutes	$\leq$ a few seconds
Offline resilience	None	store&forward buffer
Multi-node scaling	Difficult	Easy (topic pattern)

Table 4.1: Old vs new architecture comparison

Chapter 5

Implementation Plan

5.1 Task Assignment

Member	Student ID	Responsibilities
Nguyen Ngoc Duong	B22DCVT112	ESP32: sensors, TFT, FreeRTOS tasks, MQTT publish/subscribe, store&forward
Phung Dinh Dat	B22DCVT133	Cloud: MQTT Broker, Ingest Service, InfluxDB, web dashboard
Tran Van Huy	B22DCVT246	AI Service: anomaly detection, forecasting, PicoClaw, end-to-end testing

Table 5.1: Team task assignment

5.2 8-Week Timeline

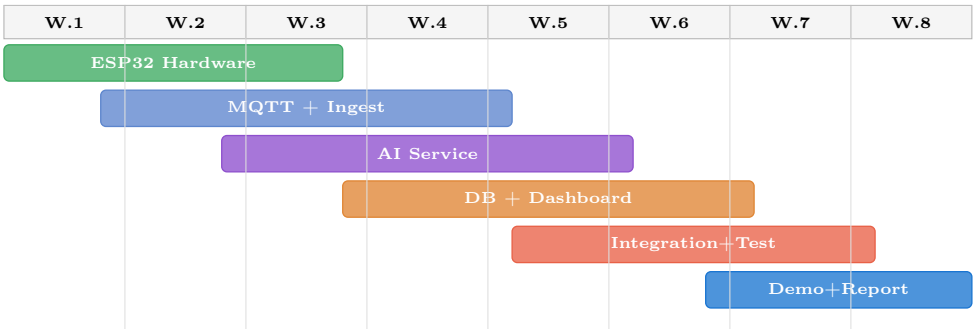

Figure 5.1: Gantt chart — 8-week implementation plan

5.3 Conclusion

The proposed system delivers a modern architecture: local safety independent of network connectivity, 2–5 second real-time streaming, industry-standard MQTT protocol, store&forward to prevent data loss, and an AI Service with anomaly detection, forecasting, and natural language reasoning. The system is practically applicable in homes, schools, offices, and small workshops.

Instructor: Nguyen Chien Trinh

Nguyen Ngoc Duong B22DCVT112 | **Phung Dinh Dat** B22DCVT133 | **Tran Van Huy** B22DCVT246